Linux Kernel Programming

Linux Kernel programming

Agenda

- GPIO Driver
- GPIO Using Sysfs
- GPIO controller Driver – Gpio Chip
- GPIO Driver
- gpiod example
- Viewing all allocated interrupt (IRQ) lines
- Interrupt sharing
- PinCtrl
- pwm

GPIO using sysfs

- The GPIOs are represented on Raspberry Pi as files and can be found in the directory tree under “/ sys / class / gpio”. This is called “sysfs”.

```
ls /sys/class/gpio
```

- When controlling and programming you have to make sure that you do not choose the pin number, but the BCM number of the GPIO. In the following example, we configure the GPIO 19 (pin 35) as the output.

- To configure a GPIO you have to create it first. Then you decide on the direction. So either as input or output. First, we turn on the “GPIO 19”.

```
echo 19> /sys/class/gpio/export
```

GPIO using sysfs

- If the GPIO is to be configured as an input, the following command applies.

  echo in> /sys/class/gpio/gpio19/direction

- If the GPIO is to be configured as an output, the following command applies.

  echo out> /sys/class/gpio/gpio19/direction

- A GPIO configured as an output should then be given a status, ie “high” or “low”.

- Basically you can only set outputs. To set the state of a GPIO output to “high”, the following command is sufficient.

  echo 1> /sys/class/gpio/gpio19/value

GPIO using sysfs

- To set the state of a GPIO output to “low” the following command is sufficient.

  echo 0> /sys/class/gpio/gpio19/value

- To determine the state, ie “high” or “low”, at a GPIO, the following command is sufficient. The GPIO can be both an input and an output.

  cat /sys/class/gpio/gpio17/value

- The output is “1” for “high” or “0” for “low”.

GPIO using sysfs

- If you no longer need GPIOs, then you should disable them. The following commands will cause the files and directories of the GPIOs to disappear. When restarting, the GPIOs are automatically reset.

echo 19> /sys/class/gpio/unexport

GPIO Controller Driver

- GPIO Controller Drivers for such devices should provide the following:
  - Methods to establish GPIO direction (input and output).
  - Methods used to access GPIO values (get and set).
  - Methods to map a given GPIO to IRQ and return the associated number.
  - A flag saying whether calls to its methods may sleep. This is very important.
  - An optional debugfs dump method (showing extra state such as pullup config).
  - An optional number called a base number, from which GPIO numbering should start. It will be automatically assigned if omitted.

GPIO Controller Driver

- In the kernel, a GPIO controller is represented as an instance of struct gpio_chip, defined in linux/gpio/driver.h:

```
struct gpio_chip {
  const char *label;
  struct device *dev;
  struct module *owner;
  int (*request)(struct gpio_chip *chip, unsigned offset);
  void (*free)(struct gpio_chip *chip, unsigned offset);
  int (*get_direction)(struct gpio_chip *chip, unsigned offset);
  int (*direction_input)(struct gpio_chip *chip, unsigned offset);
  int (*direction_output)(struct gpio_chip *chip, unsigned offset,
        int value);
```

GPIO Controller Driver

```
        int (*get)(struct gpio_chip *chip,unsigned offset);
void (*set)(struct gpio_chip *chip, unsigned offset, int value);
void (*set_multiple)(struct gpio_chip *chip, unsigned long *mask,
        unsigned long *bits);
int (*set_debounce)(struct gpio_chip *chip, unsigned offset,
        unsigned debounce);

int (*to_irq)(struct gpio_chip *chip, unsigned offset);

int base;
u16 ngpio;
const char *const *names;
bool can_sleep;
bool irq_not_threaded;
bool exported;
```

GPIO Controller Driver

```
#ifdef CONFIG_GPIOLIB_IRQCHIP
  /*
   * With CONFIG_GPIOLIB_IRQCHIP we get an irqchip
    * inside the gpiolib to handle IRQs for most practical cases.
   */
  struct irq_chip *irqchip;
  struct irq_domain *irqdomain;
  unsigned int irq_base;
  irq_flow_handler_t  irq_handler;
  unsigned int irq_default_type;
#endif
```

GPIO Controller Driver

```
#if defined(CONFIG_OF_GPIO)
  /*
   * If CONFIG_OF is enabled, then all GPIO controllers described in the
   * device tree automatically may have an OF translation
   */
  struct device_node *of_node;
  int of_gpio_n_cells;
  int (*of_xlate)(struct gpio_chip *gc,
      const struct of_phandle_args *gpiospec, u32 *flags);
#endif
}
```

GPIO Controller Driver

- The following is the meaning of each element in the structure:
  - **request** is an optional hook for chip-specific activation. If provided, it is executed prior to allocating GPIO whenever you call gpio_request() or gpiod_get().
  - **free** is an optional hook for chip-specific deactivation. If provided, it is executed before the GPIO is deallocated whenever you call gpiod_put() or gpio_free().
  - **get_direction** is executed whenever you need to know the direction of the GPIO offset. Return value should be 0 to mean out, and 1 to mean in(the same as GPIOF_DIR_XXX), or negative error.
  - **direction_input** configures the signal offset as input, or returns an error.
  - **get** returns the value of GPIO offset; for output signals, this returns either the value actually sensed or zero.
  - **set** assigns an output value to the GPIO offset.
  - **set_multiple** is called when you need to assign output values for multiple signals defined by mask. If not provided, the kernel will install a generic hook that will walk through mask bits and execute chip->set(i) on each bit set.

GPIO Controller Driver

- **set_debounce** if supported by the controller, this hook is an optional callback provided to set the debounce time for the specified GPIO.
- **to_irq** is an optional hook to provide GPIO to IRQ mapping. This is called whenever you want to execute the gpio_to_irq() or gpiod_to_irq() function. This implementation may not sleep.
- **base** identifies the first GPIO number handled by this chip; or, if negative during registration, the kernel will automatically (dynamically) assign one.
- **ngpio** is the number of GPIOs this controller provides; it starts from base to (base + ngpio - 1).
- **names**, if set, must be an array of strings to use as alternative names for the GPIOs in this chip. The array must be ngpio sized, and any GPIO that does not need an alias may have its entry set to NULL in the array.
- **can_sleep** is a Boolean flag to be set if the get()/set() method may sleep. It is the case for the GPIO controller (also known as an expander) sitting on a bus, such as I2C or SPI, whose access may lead to sleep. This implies that, if the chip supports IRQs, these IRQs need to be threaded as the chip access may sleep when, for example, reading out the IRQ status registers. For a GPIO controller mapped to memory (part of SoC), this can be set to false.
- **irq_not_threaded** is a Boolean flag and must be set if can_sleep is set, but the IRQs don't need to be threaded.

GPIO Controller Driver

- After every callback has been defined and other fields set, you should call gpiochip_add() on the configured struct gpio_chip structure in order to register the controller with the kernel. When it comes to unregistering, use gpiochip_remove().

GPIO Driver

- Already Kernel GPIO subsystems have provided the APIs to access the GPIOs.

```
#include <linux/gpio.h>
```

GPIO Driver - Validate the GPIO

- Before using the GPIO, we must check the GPIO that we are planning to use is valid or not. To do that we have to use the below API.

bool gpio_is_valid(int gpio_number);

Where,

gpio_number : GPIO that you are planning to use

- It returns false if it is not valid otherwise, it returns true.

- This API determines whether the GPIO number is valid or not. Sometimes this call will return false even if you send a valid number. Because that GPIO pin might temporarily unused on a given board.

GPIO Driver - Request the GPIO

- Once you have validated the GPIO, then you can request the GPIO using the below APIs.

int gpio_request(unsigned gpio, const char *label)

where,

gpio : GPIO that you are planning to use.

label : label used by the kernel for the GPIO in sysfs. You can provide any string that can be seen in /sys/kernel/debug/gpio. Do cat /sys/kernel/debug/gpio. You can see the GPIO assigned to the particular GPIO.

- It returns 0 in success and a negative number in failure.

Note: GPIOs must be allocated before use.

GPIO Driver - Request the GPIO

- There are other variants also available. You can use any one of them based on your need.

int gpio_request_one(unsigned gpio, unsigned long flags, const char *label); – Request one GPIO.

int gpio_request_array(struct gpio *array, size_t num); – Request multiple GPIOs.

GPIO Driver - Export the GPIO

- For debugging purposes, you can export the GPIO which is allocated using the gpio_request()to the sysfs using the below API.

int gpio_export(unsigned int gpio, bool direction_may_change);

where,

gpio : GPIO that you want to export.

direction_may_change : This parameter controls whether user space is allowed to change the direction of the GPIO. true – Can change, false – Can't change.

- Returns zero on success, else an error.

- Once you export the GPIO, you can see the GPIO in /sys/class/gpio/*. There you can change the GPIO's value, etc.

GPIO Driver – Unexport the GPIO

- If you have exported the GPIO using the gpio_export(), then you can unexport this using the below API.

void gpio_unexport(unsigned int gpio)

Where,

gpio : GPIO that you want to unexport

GPIO Driver – Set the direction of the GPIO

- When you are working on the GPIOs, you have set the GPIO as output or input. The below APIs are used to achieve that.

- This API is used to set the GPIO direction as input.

  int gpio_direction_input(unsigned gpio)

Where,

gpio : GPIO that you want to set the direction as input.

- Returns zero on success, else an error.

GPIO Driver – Set the direction of the GPIO

- This API is used to set the GPIO direction as output.

  int gpio_direction_output(unsigned gpio, int value)

Where,

gpio : GPIO that you want to set the direction as output.

value : The value of the GPIO once the output direction is effective.

- Returns zero on success, else an error.

GPIO Driver – Change the GPIO value

- Once you set the GPIO direction as an output, then you can use the below API to change the GPIO value.

gpio_set_value(unsigned int gpio, int value);

where,

gpio : GPIO that you want to change the value.

value : value to set to the GPIO. 0 – Low, 1 – High.

GPIO Driver – Read the GPIO value

- You can read the GPIO's value using the below API.

```
int  gpio_get_value(unsigned gpio);
```

where,

gpio : GPIO that you want to read the value.

- It returns the GPIO's value. 0 or 1.

GPIO Driver – Release the GPIO

- Once you have done with the GPIO, then you can release the GPIO which you have allocated previously. The below API help in that case.

void gpio_free(unsigned int gpio);

where,

gpio : GPIO that you want to release.

GPIO Driver – Release the GPIO

- There are other variants also available. You can use any one of them based on your need.

void gpio_free_array(struct gpio *array, size_t num); – Release multiple GPIOs.

GPIO Driver with Interrupt

- Accessing the input GPIO in Linux Kernel with interrupt enabled:
  - Verify the GPIO is valid or not.
  - If it is valid, then you can request the GPIO from the Kernel GPIO subsystem.
  - Set the direction of the GPIO as an input
  - Set the debounce-interval
  - Read the GPIO.
  - You enable IRQ also for edge/level triggered if you need it.
  - Then release the GPIO while exiting the driver or once you are done.

GPIO Driver with Interrupt

Set the debounce-interval

- The below API is used for sets debounce time for a GPIO.

  int gpiod_set_debounce(unsigned gpio, unsigned debounce);

where,

gpio : GPIO that you want to set the debounce value.

debounce : Delay of the debounce.

- It returns 0 on success. otherwise, <0.

GPIO Driver with Interrupt

Get the IRQ number for the GPIO

- Using the below API, you can get the IRQ number for the specific GPIO.

int gpio_to_irq(unsigned gpio);

where,

gpio : GPIO that you want to get the IRQ number.

GPIO Driver with Interrupt

Request the IRQ

- Here where you need to register the GPIO IRQ number and its handler to the Linux Interrupts with the proper interrupt flags. You can find the details regarding requesting the GPIO here. Please note that you have to free the IRQ once you are done with the interrupt.

Interrupt Flags

- While registering the GPIO interrupt using request_irq() you can use any one of the flags based on your need.

  - IRQF_TRIGGER_RISING
  - IRQF_TRIGGER_FALLING
  - IRQF_TRIGGER_HIGH
  - IRQF_TRIGGER_LOW

gpiod example

- Example

Viewing all allocated interrupt (IRQ) lines

```
rpi0w ~ $ cat /proc/interrupts
           CPU0
 17:       1035  ARMCTRL-level   1 Edge      2000b880.mailbox
 18:         36  ARMCTRL-level   2 Edge      VCHIQ doorbell
 27:      75794  ARMCTRL-level  35 Edge      timer
 40:          0  ARMCTRL-level  48 Edge      bcm2708_fb DMA
 42:       1251  ARMCTRL-level  50 Edge      DMA IRQ
 44:       5652  ARMCTRL-level  52 Edge      DMA IRQ
 56:          1  ARMCTRL-level  64 Edge      dwc_otg, dwc_otg_pcd, dwc_otg_hcd:usb1
 80:       1166  ARMCTRL-level  88 Edge      mmc0
 81:       4145  ARMCTRL-level  89 Edge      uart-pl011
 86:     113854  ARMCTRL-level  94 Edge      mmc1
FIQ:              usb_fiq
Err:          0
rpi0w ~ $
```

- In the preceding /proc/interrupts output, one line (or record) is emitted for each IRQ line on the system. Let's interpret each column of the output:
  - The first column is the IRQ number that's been allocated.
  - The second column (onward) shows the number of hardirqs that have been serviced by each CPU core (from system startup until now). The number represents the number of times the interrupt handler ran on that CPU core (the number of columns varies, depending on the number of active cores that are handling IRQs on the system). In the preceding screenshot, the Raspberry Pi Zero has only one CPU core, whereas our x86_64 VM has two (virtualized) CPU cores that interrupts are distributed over and handled.

Viewing all allocated interrupt (IRQ) lines

- The third (or later) column shows the interrupt controller chip. On x86 (the fourth column in Figure 4.9), the name IO-APIC means that the interrupt controller is an enhanced one that's used on multicore systems to distribute interrupts to various cores or CPU groups (on high-end systems, multiple IOAPICs may be in play).
- The column after that displays the type of interrupt triggering that's being used; that is, level or edge triggering (we discussed this in the Understanding level- and edge-triggered interrupts section). Here, Edge tells us that the IRQ is edgetriggered. The number that's prefixed to it (for example, 35 Edge in the preceding screenshot) is very system-dependent. It often represents the interrupt source (that the kernel maps to an IRQ line; many embedded device drivers often use GPIO pins to serve as interrupt sources). It's best not to attempt to interpret it (unless you actually know how to) and just rely on the IRQ number instead (the first column)
- The last column on the right states the current owner of the IRQ line. Typically, this is the name of the device driver or kernel component (that allocated this IRQ line via one of the *request_*irq() APIs).

Viewing all allocated interrupt (IRQ) lines

- The third (or later) column shows the interrupt controller chip. On x86 (the fourth column in Figure 4.9), the name IO-APIC means that the interrupt controller is an enhanced one that's used on multicore systems to distribute interrupts to various cores or CPU groups (on high-end systems, multiple IOAPICs may be in play).
- The column after that displays the type of interrupt triggering that's being used; that is, level or edge triggering (we discussed this in the Understanding level- and edge-triggered interrupts section). Here, Edge tells us that the IRQ is edgetriggered. The number that's prefixed to it (for example, 35 Edge in the preceding screenshot) is very system-dependent. It often represents the interrupt source (that the kernel maps to an IRQ line; many embedded device drivers often use GPIO pins to serve as interrupt sources). It's best not to attempt to interpret it (unless you actually know how to) and just rely on the IRQ number instead (the first column)
- The last column on the right states the current owner of the IRQ line. Typically, this is the name of the device driver or kernel component (that allocated this IRQ line via one of the *request_*irq() APIs).

Viewing all allocated interrupt (IRQ) lines

```
vivekananda@vivekananda-Inspiron-5570:~$ cat /proc/interrupts
           CPU0       CPU1       CPU2       CPU3       CPU4       CPU5       CPU6       CPU7
  0:          9          0          0          0          0          0          0          0  IR-IO-APIC    2-edge      timer
  1:          0          0          0          0          0          0          0       1941  IR-IO-APIC    1-edge      i8042
  8:          0          1          0          0          0          0          0          0  IR-IO-APIC    8-edge      rtc0
  9:          0       3783          0          0          0          0          0          0  IR-IO-APIC    9-fasteoi   acpi
 12:          0          0          0          0          0          0        177          0  IR-IO-APIC   12-edge      i8042
 14:          0          0          0          0          0          0          0          0  IR-IO-APIC   14-fasteoi   INT344B:00
 16:          0          0          0          0    1176486          0          0          0  IR-IO-APIC   16-fasteoi   idma64.0, i2c_designware.0
 51:          0          0          0          0          0      38624          0          0  IR-IO-APIC   51-fasteoi   DELL0810:00
120:          0          0          0          0          0          0          0          0  DMAR-MSI    0-edge      dmar0
121:          0          0          0          0          0          0          0          0  DMAR-MSI    1-edge      dmar1
122:          0          0          0          0          0          0          0          0  IR-PCI-MSI 458752-edge      aerdrv, PCIe PME
123:          0          0          0          0          0          0          0          0  IR-PCI-MSI 466944-edge      aerdrv, PCIe PME
124:          0          0          0          0          0          0          0          0  IR-PCI-MSI 468992-edge      PCIe PME
125:          0          0          0        100          0        338          0          0  IR-PCI-MSI 327680-edge      xhci_hcd
126:          0          0     349454       8362          0          0          0          0  IR-PCI-MSI 376832-edge      ahci[0000:00:17.0]
127:          0          0          0          0          0          0          0          0  IR-PCI-MSI 1048576-edge      enp2s0
128:          0          0          0          0          0          0         36          0  IR-PCI-MSI 360448-edge      mei_me
129:          0          0          0          0          0          0          0       1686  IR-PCI-MSI 514048-edge      snd_hda_intel:card0
130:          0        261          0          0      87749          0          0          0  IR-PCI-MSI 1572864-edge      ath10k_pci
NMI:         19         19         22         19         20         18         19         18   Non-maskable interrupts
LOC:      93580      88863     202621     108234     126052      93680      89349      89103   Local timer interrupts
SPU:          0          0          0          0          0          0          0          0   Spurious interrupts
PMI:         19         19         22         19         20         18         19         18   Performance monitoring interrupts
IWI:          0          1          0          0          0          0          1          0   IRQ work interrupts
RTR:          0          0          0          0          0          0          0          0   APIC ICR read retries
RES:      46873      34480      15007      12504      16663      11076      10612      10483   Rescheduling interrupts
CAL:      61864      79485      68875      53967      58465      59484      80335      67679   Function call interrupts
TLB:      57621      76949      66625      51597      56193      57080      77933      65181   TLB shootdowns
TRM:         46         46         46         46         46         46         46         46   Thermal event interrupts
THR:          0          0          0          0          0          0          0          0   Threshold APIC interrupts
DFR:          0          0          0          0          0          0          0          0   Deferred Error APIC interrupts
MCE:          0          0          0          0          0          0          0          0   Machine check exceptions
MCP:          7          8          8          8          8          8          8          8   Machine check polls
HYP:          0          0          0          0          0          0          0          0   Hypervisor callback interrupts
ERR:          0
MIS:          0
PIN:          0          0          0          0          0          0          0          0   Posted-interrupt notification event
NPI:          0          0          0          0          0          0          0          0   Nested posted-interrupt event
PIW:          0          0          0          0          0          0          0          0   Posted-interrupt wakeup event
```

Viewing all allocated interrupt (IRQ) lines

```
$ cat /proc/interrupts
           CPU0       CPU1
  0:         35          0   IO-APIC   2-edge      timer
  1:          9          0   IO-APIC   1-edge      i8042
  4:          0        672   IO-APIC   4-edge      ttyS0
  8:          0          0   IO-APIC   8-edge      rtc0
  9:          0          0   IO-APIC   9-fasteoi   acpi
 12:          0        158   IO-APIC  12-edge      i8042
 14:          0          0   IO-APIC  14-edge      ata_piix
 15:          0       2230   IO-APIC  15-edge      ata_piix
 16:         69       9768   IO-APIC  16-fasteoi   enp0s8
 18:        420         21   IO-APIC  18-fasteoi   vmwgfx
 19:       1049        225   IO-APIC  19-fasteoi   enp0s3
 21:      42670          0   IO-APIC  21-fasteoi   ahci[0000:00:0d.0], snd_intel8x0
 22:         26          0   IO-APIC  22-fasteoi   ohci_hcd:usb1
NMI:          0          0   Non-maskable interrupts
LOC:    1152560    2011317   Local timer interrupts
SPU:          0          0   Spurious interrupts
PMI:          0          0   Performance monitoring interrupts
IWI:          0          0   IRQ work interrupts
```

- From the 2.6.24 kernel, for x86 and AMD64 systems (or x86_64), even non-device (I/O) interrupts (system interrupts) are displayed here, such as the NMI, local timer interrupt (LOC), PMI, IWI, and so on. You can see in Figure , the last line displays IWI, which is the Inter-Work Interrupt.

- The kernel procfs code that displays the preceding output of /proc/interrupts – that is, its show method – can be found at kernel/irq/proc.c:show_interrupts()

(link: https://elixir.bootlin.com/linux/v5.4/source/kernel/irq/proc.c#L438) .

Viewing all allocated interrupt (IRQ) lines

- First, it prints the header line, then emits a one-line "record" for each IRQ line. The statistics are mainly obtained from within the metadata structure for each IRQ line – struct irq_desc; within each IRQ, it loops over every processor core (via the for_each_online_cpu() helper routine), printing the number of hardirqs that have been served for each of them. Finally (last column), it prints the "owner" of the IRQ line via the name member of struct irqaction. The arch-specific interrupts for the x86 (such as the NMI, LOC, PMI, and IWI IRQs) are displayed via the codeat arch/x86/kernel/irq.c:arch_show_interrupts().

- Each of the types of interrupts seen in this file, which are architecture-specific, mean something different. For x86 machines, the following values are common:
  - XT-PIC — This is the old AT computer interrupts.
  - IO-APIC-edge — The voltage signal on this interrupt transitions from low to high, creating an edge, where the interrupt occurs and is only signaled once. This kind of interrupt, as well as the IO-APIC-level interrupt, are only seen on systems with processors from the 586 family and higher.
  - IO-APIC-level — Generates interrupts when its voltage signal is high until the signal is low again.

- /proc/softirqs;

Interrupt Sharing

- The notion of an IRQ conflict is almost synonymous with the PC architecture. In general, IRQ lines on the PC have not been able to serve more than one device, and there have never been enough of them. As a result, frustrated users have often spent much time with their computer case open, trying to find a way to make all of their hardware play well together.

- But, in fact, there is nothing in the design of the hardware itself that says that interrupt lines cannot be shared. The problems are on the software side. With the arrival of the PCI bus, the writers of system software have had to work a little harder, since all PCI interrupts can explicitly be shared. So Linux supports shared interrupts—and on all buses where it makes any sense, not just the PCI. Thus, suitably aware drivers for ISA devices can also share an IRQ line.

- The question of interrupt sharing under the ISA bus brings in the issue of level-triggered versus edge-triggered interrupt lines. Although the former kind of interrupt reporting is safe with regard to sharing, it may lead to software lockup if not handled correctly. Edge-triggered interrupts, on the other hand, are not safe with regard to sharing; ISA is edge triggered, because this signaling is easier to implement at hardware level and therefore was the common choice in the 1980s. This issue is unrelated to electrical signal levels; in order to support sharing, the line must be able to be driven active by multiple sources whether it is level triggered or edge triggered.

Interrupt Sharing

- With a level-triggered interrupt line, the peripheral device asserts the IRQ signal until software clears the pending interrupt (usually by writing to a device register); therefore, if several devices pull the line active, the CPU will signal an interrupt as soon as the IRQ is enabled until all drivers have serviced their devices. This behavior is safe with regard to sharing but may lead to lockup if a driver fails to clear its interrupt source.

- When using edge-triggered interrupts, on the other hand, interrupts may be lost: if one device pulls the line active for too long a time, when another device pulls the line active no edge will be generated, and the processor will ignore the second request. A shared handler may just not see the interrupt, and if its hardware doesn't deassert the IRQ line no other interrupt will be notified for either shared device.

- For this reason, even if interrupt sharing is supported under ISA, it may not function properly; while some devices pull the IRQ line active for a single clock cycle, other devices are not so well behaved and may cause great pains to the driver writer who tries to share the IRQ. We won't go any deeper into this issue; for the rest of this section we assume that either the host bus supports sharing or that you know what you are doing.

Interrupt Sharing

- To develop a driver that can manage a shared interrupt line, some details need to be considered. As discussed later, some of the features described in this chapter are not available for devices using interrupt sharing. Whenever possible, it's better to support sharing because it presents fewer problems for the final user. In some cases (e.g., when working with the PCI bus), interrupt sharing is mandatory.

Installing a Shared Handler

- Shared interrupts are installed through request_irq just like nonshared ones, but there are two differences:
  - The SA_SHIRQ bit must be specified in the flags argument when requesting the interrupt.
  - The dev_id argument must be unique. Any pointer into the module's address space will do, but dev_id definitely cannot be set to NULL.

- The kernel keeps a list of shared handlers associated with the interrupt, like a driver's signature, and dev_id differentiates between them. If two drivers were to register NULL as their signature on the same interrupt, things might get mixed up at unload time, causing the kernel to oops when an interrupt arrived. For this reason, modern kernels will complain loudly if passed a NULL dev_id when registering shared interrupts.

Interrupt Sharing

- When a shared interrupt is requested, request_irq succeeds if either the interrupt line is free or any handlers already registered for that line have also specified that the IRQ is to be shared. With 2.0 kernels, it was also necessary that all handlers for a shared interrupt were either fast or slow—the two modes could not be mixed.

- Whenever two or more drivers are sharing an interrupt line and the hardware interrupts the processor on that line, the kernel invokes every handler registered for that interrupt, passing each its own dev_id. Therefore, a shared handler must be able to recognize its own interrupts, and should quickly exit when its own device has not interrupted.

- If you need to probe for your device before requesting the IRQ line, the kernel can't help you. No probing function is available for shared handlers. The standard probing mechanism works if the line being used is free, but if the line is already held by another driver with sharing capabilities, the probe will fail, even if your driver would have worked perfectly.

Interrupt Sharing

- The only available technique for probing shared lines, then, is the do-it-yourself way. The driver should request every possible IRQ line as a shared handler and then see where interrupts are reported. The difference between that and do-it-yourself probing is that the probing handler must check with the device to see that the interrupt actually occurred, because it could have been called in response to another device interrupting on a shared line.

- Releasing the handler is performed in the normal way, using release_irq. Here the dev_id argument is used to select the correct handler to release from the list of shared handlers for the interrupt. That's why the dev_id pointer must be unique.

- A driver using a shared handler needs to be careful about one more thing: it can't play with enable_irq or disable_irq. If it does, things might go haywire for other devices sharing the line. In general, the programmer must remember that his driver doesn't own the IRQ, and its behavior should be more "social" than is necessary if one owns the interrupt line.

Interrupt Sharing

Running the Handler

- As suggested earlier, when the kernel receives an interrupt, all the registered handlers are invoked. A shared handler must be able to distinguish between interrupts that it needs to handle and interrupts generated by other devices.

- Loading short with the option shared=1 installs the following handler instead of the default:

```
void short_sh_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    int value, written;
    struct timeval tv;

    /* If it wasn't short, return immediately */
    value = inb(short_base);
    if (!(value & 0x80)) return;

    /* clear the interrupting bit */
    outb(value & 0x7F, short_base);

    /* the rest is unchanged */

    do_gettimeofday(&tv);
    written = sprintf((char *)short_head,"%08u.%06u\n",
                      (int)(tv.tv_sec % 100000000), (int)(tv.tv_usec));
    short_incr_bp(&short_head, written);
    wake_up_interruptible(&short_queue); /* wake any reading process */
}
```

Interrupt Sharing

- An explanation is due here. Since the parallel port has no “interrupt-pending” bit to check, the handler uses the ACK bit for this purpose. If the bit is high, the interrupt being reported is for short, and the handler clears the bit.

- The handler resets the bit by zeroing the high bit of the parallel interface’s data port—short assumes that pins 9 and 10 are connected together. If one of the other devices sharing the IRQ with short generates an interrupt, short sees that its own line is still inactive and does nothing.

- A full-featured driver probably splits the work into top and bottom halves, of course, but that’s easy to add and does not have any impact on the code that implements sharing. A real driver would also likely use the dev_id argument to determine which, of possibly many, devices might be interrupting.

The /proc Interface

- Installing shared handlers in the system doesn’t affect /proc/stat, which doesn’t even know about handlers. However, /proc/interrupts changes slightly.

Interrupt Sharing

- All the handlers installed for the same interrupt number appear on the same line of /proc/interrupts. The following output shows how shared interrupt handlers are displayed:

```
           CPU0       CPU1
  0:   22114216   22002860    IO-APIC-edge  timer
  1:     135401     136582    IO-APIC-edge  keyboard
  2:          0          0          XT-PIC  cascade
  5:    5162076    5160039   IO-APIC-level  eth0
  9:          0          0   IO-APIC-level  acpi, es1370
 10:     310450     312222   IO-APIC-level  aic7xxx
 12:     460372     471747    IO-APIC-edge  PS/2 Mouse
 13:          1          0          XT-PIC  fpu
 15:    1367555    1322398    IO-APIC-edge  ide1
NMI:   44117004   44117004
LOC:   44116987   44116986
ERR:          0
```

- The shared interrupt line here is IRQ 9; the active handlers are listed on one line, separated by commas. Here the power management subsystem (“acpi”) is sharing this IRQ with the sound card (“es1370”). The kernel is unable to distinguish interrupts from these two sources, and will invoke each interrupt handlers in the driver for each interrupt.

PinCtrl

- The pin control (pinctrl) subsystem allows managing pin muxing. In the DT, devices that need pins to be multiplexed in a certain way must declare the pin control configuration they need.

- The pinctrl subsystem provides:
  - Pin multiplexing, which allows for reusing the same pin for different purposes, such as one pin being a UART TX pin, GPIO line, or HSI data line. Multiplexing can affect groups of pins or individual pins.
  - Pin configuration, applying electronic properties of pins such as pull-up, pulldown, driver strength, debounce period, and so on.

- The purpose of this book is limited to using functions exported by the pin controller driver, and does not cover how to write a pin controller driver

PinCtrl

Pinctrl and the device tree

- The pinctrl subsystem is nothing but a way to gather pins (not only GPIO), and pass them to the driver. The pin controller driver is responsible for parsing pin descriptions in the DT and applying their configuration in the chip. The driver usually needs a set of two nested nodes to describe a group of pins configurations. The first node describes the function of the group (what purpose the group will be used for), the second holds the pin's configuration.

- How pin groups are assigned in the DT heavily depends on the platform, and thus the pin controller driver. Every pin control state is given an integer ID starting at 0 and contiguous. You can use a name property, which will be mapped on top of IDs, so that the same name always points to the same ID. Each client device's own binding determines the set of states that must be defined in its DT node, and whether to define the set of state IDs that must be provided or whether to define the set of state names that must be provided. In any case, a pin configuration node can be assigned to a device by means of two properties:
  - pinctrl-<ID>: This allows us to give the list of pinctrl configurations needed for a certain state of the device. It is a list of phandles, each of which points to a pin configuration node. These referenced pin configuration nodes must be child nodes of the pin controller that they configure. Multiple entries may exist in this list so that multiple pin controllers may be configured, or so that a state may be built from multiple nodes for a single pin controller, each contributing part of the overall configuration.

PinCtrl

- pinctrl-name: This allows for giving a name to each state in a list. List entry 0 defines the name for integer state ID 0, list entry 1 for state ID 1, and so on. The state ID 0 is commonly given the name default. A list of standardized states can be found in include/linux/pinctrl/pinctrl-state.h.

- The following is an excerpt of DT, showing some device nodes along with their pin control nodes:

```
usdhc@0219c000 { /* uSDHC4 */
   non-removable;
   vmmc-supply = <&reg_3p3v>;
   status = "okay";
   pinctrl-names = "default";
   pinctrl-0 = <&pinctrl_usdhc4_1>;
};

gpio-keys {
    compatible = "gpio-keys";
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_io_foo &pinctrl_io_bar>;
};
```

PinCtrl

```
iomuxc@020e0000 {
    compatible = "fsl,imx6q-iomuxc";
    reg = <0x020e0000 0x4000>;

    /* shared pinctrl settings */
    usdhc4 { /* first node describing the function */
        pinctrl_usdhc4_1: usdhc4grp-1 { /* second node */
            fsl,pins = <
                MX6QDL_PAD_SD4_CMD__SD4_CMD    0x17059
                MX6QDL_PAD_SD4_CLK__SD4_CLK    0x10059
                MX6QDL_PAD_SD4_DAT0__SD4_DATA0 0x17059
                MX6QDL_PAD_SD4_DAT1__SD4_DATA1 0x17059
                MX6QDL_PAD_SD4_DAT2__SD4_DATA2 0x17059
                MX6QDL_PAD_SD4_DAT3__SD4_DATA3 0x17059
                MX6QDL_PAD_SD4_DAT4__SD4_DATA4 0x17059
                MX6QDL_PAD_SD4_DAT5__SD4_DATA5 0x17059
                MX6QDL_PAD_SD4_DAT6__SD4_DATA6 0x17059
                MX6QDL_PAD_SD4_DAT7__SD4_DATA7 0x17059
            >;
        };
    };
    [...]
    uart3 {
        pinctrl_uart3_1: uart3grp-1 {
            fsl,pins = <
                MX6QDL_PAD_EIM_D24__UART3_TX_DATA 0x1b0b1
                MX6QDL_PAD_EIM_D25__UART3_RX_DATA 0x1b0b1
            >;
        };
    };
    // GPIOs (Inputs)
```

PinCtrl

```
    gpios {
        pinctrl_io_foo: pinctrl_io_foo {
            fsl,pins = <
                MX6QDL_PAD_DISP0_DAT15__GPIO5_IO09  0x1f059
                MX6QDL_PAD_DISP0_DAT13__GPIO5_IO07  0x1f059
            >;
        };
        pinctrl_io_bar: pinctrl_io_bar {
            fsl,pins = <
                MX6QDL_PAD_DISP0_DAT11__GPIO5_IO05  0x1f059
                MX6QDL_PAD_DISP0_DAT9__GPIO4_IO30   0x1f059
                MX6QDL_PAD_DISP0_DAT7__GPIO4_IO28   0x1f059
                MX6QDL_PAD_DISP0_DAT5__GPIO4_IO26   0x1f059
            >;
        };
    };
};
```

In the preceding example, a pin configuration is given in the form `<PIN_FUNCTION> <PIN_SETTING>`, for example:

```
MX6QDL_PAD_DISP0_DAT15__GPIO5_IO09  0x80000000
```

`MX6QDL_PAD_DISP0_DAT15__GPIO5_IO09` represents the pin function, which is GPIO in this case, and `0x80000000` represents the pin settings.

For this line:

```
MX6QDL_PAD_EIM_D25__UART3_RX_DATA 0x1b0b1
```

`MX6QDL_PAD_EIM_D25__UART3_RX_DATA` represents the pin function, which is the RX line of UART3, and `0x1b0b1` represents the settings.

The pin function is a macro whose value is meaningful for the pin controller driver only. These are generally defined in header files located in `arch/<arch>/boot/dts/`. If you use a UDOO quad, for example, which has an i.MX6 quad core (ARM), the pin function header will be `arch/arm/boot/dts/imx6q-pinfunc.h`. The following is the macro corresponding to the fifth line of the GPIO5 controller:

```
#define MX6QDL_PAD_DISP0_DAT11__GPIO5_IO05  0x19c 0x4b0 0x000 0x5 0x0
```

`<PIN_SETTING>` can be used to set up things such as pull-ups, pull-downs, keepers, drive strength, and so on. How it should be specified depends on the pin controller binding, and the meaning of its value depends on the SoC data sheet, generally in the IOMUX section. On i.MX6 IOMUXC, fewer than 17 bits are exclusively used for this purpose.

PinCtrl

These preceding nodes are called from the corresponding driver-specific node. Moreover, these pins are configured during the corresponding driver initialization. Prior to selecting a pin group state, you must get the pin control first using the `pinctrl_get()` function, call `pinctrl_lookup_state()` in order to check whether the requested state exists or not, and finally `pinctrl_select_state()` to apply the state.

The following sample shows how to get a pincontrol and apply its default configuration:

```
struct pinctrl *p;
struct pinctrl_state *s;
int ret;

p = pinctrl_get(dev);
if (IS_ERR(p))
     return p;

s = pinctrl_lookup_state(p, name);
if (IS_ERR(s)) {
     devm_pinctrl_put(p);
     return ERR_PTR(PTR_ERR(s));
}

ret = pinctrl_select_state(p, s);
if (ret < 0) {
     devm_pinctrl_put(p);
     return ERR_PTR(ret);
}
```

You usually perform such steps during driver initialization. A suitable place for this code could be within the `probe()` function.

A pin control can be released with the `pinctrl_put()` function. You can use the resource-managed version of the API. That said, you can use `pinctrl_get_select()`, given the name of the state to select, in order to configure pinmux. The function is defined in `include/linux/pinctrl/consumer.h` as follows:

```
static struct pinctrl *pinctrl_get_select(struct device *dev,
                                          const char *name)
```

PinCtrl

Here, `*name` is the state name as written in the `pinctrl-name` property. If the name of the state is `default`, you can just call the `pinctr_get_select_default()` function, which is a wrapper around `pinctl_get_select()`:

```
static struct pinctrl * pinctrl_get_select_default(
                                    struct device *dev)
{
   return pinctrl_get_select(dev, PINCTRL_STATE_DEFAULT);
}
```

Let's see a real example in a board-specific DTS file (`am335x-evm.dts`):

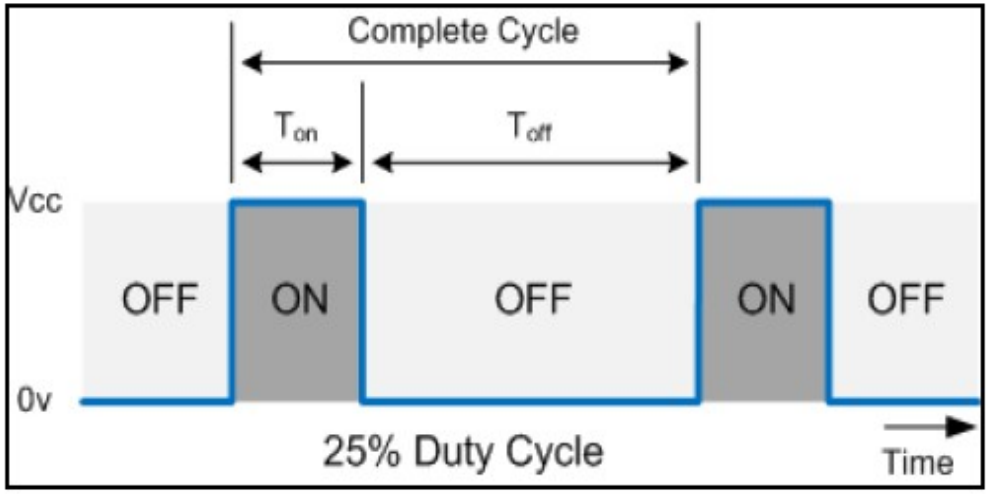
```
dcan1: d_can@481d0000 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&d_can1_pins>;
};
```

And the following is an example in the corresponding driver:

```
pinctrl = devm_pinctrl_get_select_default(&pdev->dev);
if (IS_ERR(pinctrl))
    dev_warn(&pdev->dev,"pins are not configured from the driver\n");
```

PWM

- Pulse Wide Modulation (PWM) operates like a switch that constantly cycles on and off. It is a hardware feature used to control servomotors, for voltage regulation and so on. The most well-known applications of PWM are:
  - Motor speed control
  - Light dimming
  - Voltage regulation



PWM

- The preceding figure illustrates a complete PWM cycle, introducing some terms that we need to clarify prior to getting deeper into the kernel PWM framework:
  - Ton: This is the duration during which the signal is high.
  - Toff: This is the duration during which the signal is low.
  - Period: This is the duration of a complete PWM cycle. It represents the sum of the Ton and Toff of the PWM signal.
  - Duty cycle: This is represented as a percentage of the time signal that remains on during the period of the PWM signal.

- The different formulas are detailed as follows:

  - PWM period: $Ton + Toff$
  - Duty cycle: $D = \frac{Ton}{Ton+Toff} \times 100 = \frac{Ton}{Period} \times 100$

PWM

- The preceding figure illustrates a complete PWM cycle, introducing some terms that we need to clarify prior to getting deeper into the kernel PWM framework:
  - Ton: This is the duration during which the signal is high.
  - Toff: This is the duration during which the signal is low.
  - Period: This is the duration of a complete PWM cycle. It represents the sum of the Ton and Toff of the PWM signal.
  - Duty cycle: This is represented as a percentage of the time signal that remains on during the period of the PWM signal.

- The different formulas are detailed as follows:

  - PWM period: $Ton + Toff$
  - Duty cycle: $D = \frac{Ton}{Ton+Toff} \times 100 = \frac{Ton}{Period} \times 100$

PWM

- The Linux PWM framework has two interfaces:
  - Controller interface: The one that exposes the PWM line. It is the PWM chip, that is, the producer.
  - Consumer interface: The device consuming PWM lines exposed by the controller. The drivers of such devices use helper functions exported by the controller by means of a generic PWM framework.

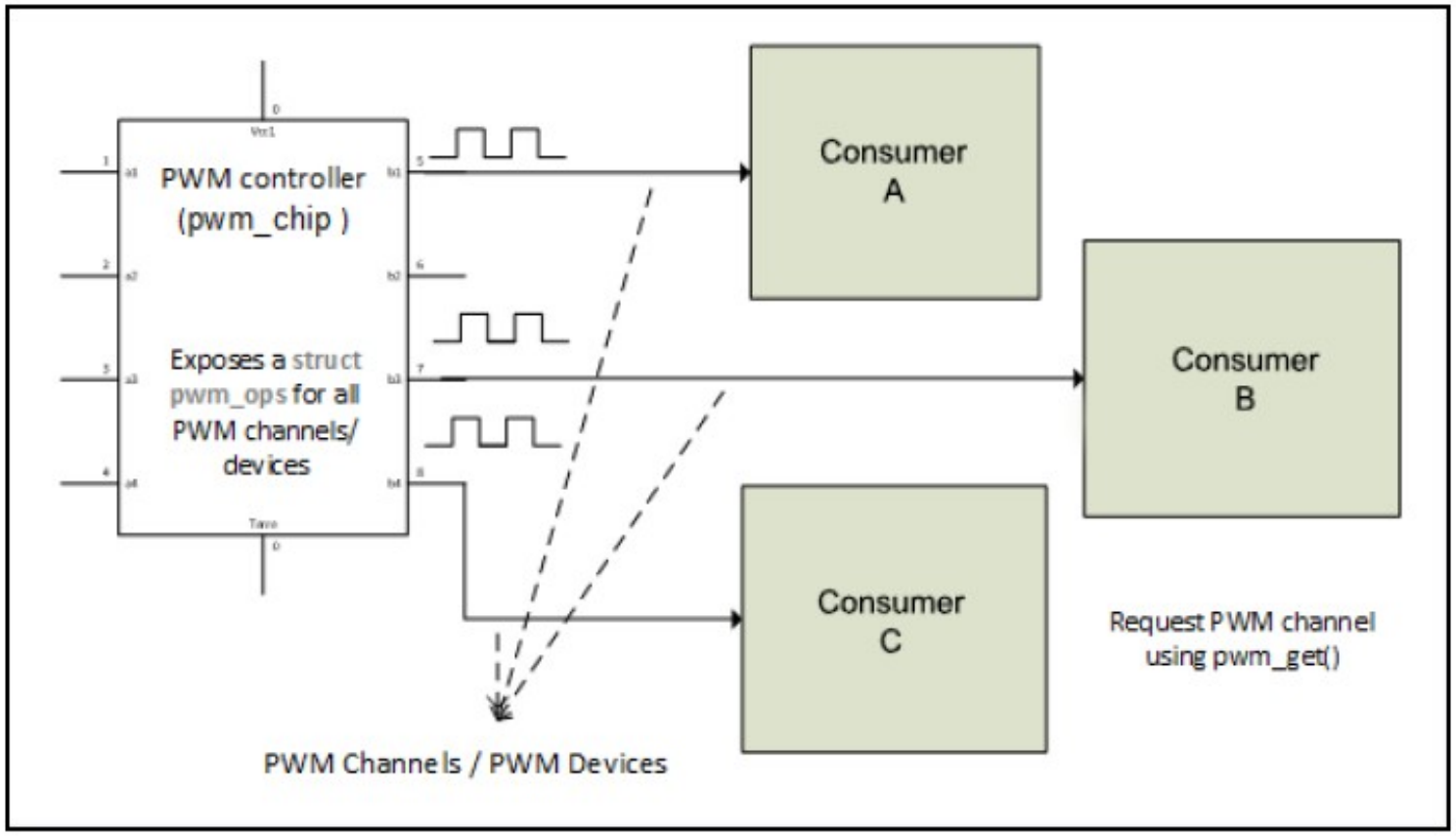
- Either the consumer or producer interface depends on the following header file:

```
#include <linux/pwm.h>
```

PWM

PWM controller driver

As you need struct gpio_chip when writing GPIO-controller drivers and struct irq_chip when writing IRQ-controller drivers, a PWM controller is represented in the kernel as an instance of the struct pwm_chip structure:



PWM

PWM controller driver

As you need struct gpio_chip when writing GPIO-controller drivers and struct irq_chip when writing IRQ-controller drivers, a PWM controller is represented in the kernel as an instance of the struct pwm_chip structure:

```
struct pwm_chip {
  struct device *dev;
  const struct pwm_ops *ops;
  int base;
  unsigned int npwm;
  struct pwm_device *pwms;
  struct pwm_device * (*of_xlate)(struct pwm_chip *pc,
                const struct of_phandle_args *args);
  unsigned int of_pwm_n_cells;
  bool can_sleep;
};
```

PWM

- The following lists the meaning of each element in the structure:
  - **dev:** This represents the device associated with the chip.
  - **Ops:** This is a data structure providing callback functions this chip exposes to consumer drivers.
  - **Base:** This is the number of the first PWM controlled by this chip. If chip->base< 0, then the kernel will dynamically assign a base. can_sleep: This should be set to true by the chip driver if the .config(), .enable(), or .disable() operations of the ops field might sleep.
  - **npwm:** This is the number of PWM channels (devices) that this chip provides.
  - **pwms:** This is an array of the PWM devices of this chip, allocated by the framework to consumer drivers.
  - **of_xlate:** This is an optional callback to request a PWM device, given a DT PWM specifier. If not defined, it will be set to of_pwm_simple_xlate by the PWM core, which will force of_pwm_n_cells to 2, as well.
  - **of_pwm_n_cells:** This is the number of cells expected in the DT for a PWM specifier.

PWM

- PWM controller/chip addition and removal rely on two basic functions: pwmchip_add() and pwmchip_remove(). Each function should be given a filled in struct pwm_chip structure as an argument. Their respective prototypes are as follows:

  int pwmchip_add(struct pwm_chip *chip)

  int pwmchip_remove(struct pwm_chip *chip)

- Unlike other framework removal functions, which do not have return values, pwmchip_remove()has a return value. It returns 0 up on success, or -EBUSY if the chip has a PWM line still in use (still requested).

- Each PWM driver must implement some hooks through the struct pwm_ops field, which is used by the PWM core or the consumer interface to configure and make full use of its PWM channels. Some of them are optional:

PWM

```
struct pwm_ops {
   int (*request)(struct pwm_chip *chip, struct pwm_device *pwm);
   void (*free)(struct pwm_chip *chip, struct pwm_device *pwm);
   int (*config)(struct pwm_chip *chip, struct pwm_device *pwm,
                 int duty_ns, int period_ns);
   int (*set_polarity)(struct pwm_chip *chip, struct pwm_device *pwm,
                 enum pwm_polarity polarity);
   int (*enable)(struct pwm_chip *chip,struct pwm_device *pwm);
   void (*disable)(struct pwm_chip *chip, struct pwm_device *pwm);
   void (*get_state)(struct pwm_chip *chip, struct pwm_device *pwm,
            struct pwm_state *state); /* since kernel v4.7 */
   struct module *owner;
};
```

PWM

- Let's look at what each element in the structure means:
  - **request:** This is an optional hook that, if provided, is executed during a PWM channel request.
  - **free:** This is the same as request; runs during PWM freeing.
  - **config:** This is the PMW configuration hook. It configures duty cycles and period length for this PWM.
  - **set_polarity:** This hook configures the polarity of this PWM.
  - **Enable:** This enables the PWM line, starting output toggling.
  - **Disable:** This disables the PWM line, stopping output toggling.
  - **Apply:** This automatically applies a new PWM config. The state argument should be adjusted with the real hardware config.
  - **get_state:** This returns the current PWM state. This function is only called once per PWM device, when the PWM chip is registered.
  - **Owner:** This is the module that owns the chip, usually THIS_MODULE.

PWM

- In the probe function of the PWM controller driver, it is a good practice to retrieve DT resources, initialize hardware, fill a **struct pwm_chip** and its **struct pwm_ops**, and then add the PWM chip with the **pwmchip_add** function.

PWM

PWM controller binding

- While binding the PWM controller from within the DT, the most important property is #pwm-cells. It represents the number of cells used to represent a PWM device on this controller. If you remember, in the struct pwm_chip structure, the of_xlate hook is used to translate a given PWM specifier. If the hook has not been set, pwm-cells must be set to 2; otherwise, it should be set with the same value as of_pwm_n_cells. The following is an example of a PWM controller node in the DT for an i.MX6 SoC:

```
pwm3: pwm@02088000 {
    #pwm-cells = <2>;
    compatible = "fsl,imx6q-pwm", "fsl,imx27-pwm";
    reg = <0x02088000 0x4000>;
    interrupts = <0 85 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&clks IMX6QDL_CLK_IPG>,
        <&clks IMX6QDL_CLK_PWM3>;
    clock-names = "ipg", "per";
    status = "disabled";
};
```

PWM

- On the other hand, the node that corresponds to our fake-pwm driver looks like this:

```
fake_pwm: pwm@0 {
    #pwm-cells = <2>;
    compatible = “test,fake-pwm";
    /*
     * Our driver does not use resource
     * neither mem, IRQ, nor Clock)
     */
};
```

PWM

Enabling hardware PWM

- In order to have the hardware PWM activated on the Raspberry Pi, you'll have to edit the /boot/config.txt file and add an overlay.

- Main Raspberry Pi kernel documentation gives 2 possibilities. Either a single channel, either a dual channel.

- Here are the possible options for each PWM channel:

PWM	GPIO	Function	Alt	Exposed
PWM0	12	4	Alt0	Yes
PWM0	18	2	Alt5	Yes
PWM0	40	4	Alt0	No
PWM0	52	5	Alt1	No
PWM1	13	4	Alt0	Yes
PWM1	19	2	Alt5	Yes
PWM1	41	4	Alt0	No
PWM1	45	4	Alt0	No
PWM1	53	5	Alt1	No

PWM

- Only accessible pin from this list on the Raspberry Pi pin out are GPIO 12, 18, 13 and 19. The other GPIO are not exposed.

Activating only 1 channel

- We have then 4 options for the exposed GPIO pins:

PWM	GPIO	Function	Alt	dtoverlay
PWM0	12	4	Alt0	dtoverlay=pwm,pin=12,func=4
PWM0	18	2	Alt5	dtoverlay=pwm,pin=18,func=2
PWM1	13	4	Alt0	dtoverlay=pwm,pin=13,func=4
PWM1	19	2	Alt5	dtoverlay=pwm,pin=19,func=2

- Edit the /boot/config.txt file and add the dtoverlay line in the file. You need root privileges for this:

  sudo nano /boot/config.txt

PWM

- Save the file with ctrl + x then Y then enter

- Then reboot:

  sudo reboot

- You are all setup, the basic example should now work with the PWM and channel you have selected.

Activating 2 channels

PWM0	PWM0 GPIO	PWM0 Function	PWM0 Alt	PWM1	PWM1 GPIO	PWM1 Function	PWM1 Alt	dtoverlay
PWM0	12	4	Alt0	PWM1	13	4	Alt0	dtoverlay=pwm-2chan,pin=12,func=4,pin2=13,func2=4
PWM0	18	2	Alt5	PWM1	13	4	Alt0	dtoverlay=pwm-2chan,pin=18,func=2,pin2=13,func2=4
PWM0	12	4	Alt0	PWM1	19	2	Alt5	dtoverlay=pwm-2chan,pin=12,func=4,pin2=19,func2=2
PWM0	18	2	Alt5	PWM1	19	2	Alt5	dtoverlay=pwm-2chan,pin=18,func=2,pin2=19,func2=2

PWM

- Edit the /boot/config.txt file and add the dtoverlay line in the file. You need root privileges for this:

  sudo nano /boot/config.txt

- Save the file with ctrl + x then Y then enter

- Then reboot:

  sudo reboot

- You are all setup, the basic example should now work with the PWM and channel you have selected.

PWM

```
# fdtdump pwm.dtbo
# dtoverlay pwm
# cd /sys/class/pwm/
# cd pwmchip0
# cat npwm
# echo 0 > export
# cd pwm0/
# echo 10000000 > period
# echo 8000000 > duty_cycle (80% dutycycle)
# echo 1 > enable
# echo 0 > enable
# echo 80000 > duty_cycle (8% dutycycle)
# echo 1 > enable
```

PWM

PWM client binding

- PWM devices can be assigned to the consumer from:
  - The device tree
  - ACPI
  - Static lookup tables, in the board init file

- DT binding is the recommended method. When binding a PWM consumer (client) to its driver, you need to provide the phandle of the controller to which it is linked. It is recommended that you give the name pwms to PWM properties; since PWM devices are named resources, you can provide an optional property, pwm-names, containing a list of strings, to name each of the PWM devices listed in the pwms property. If no pwm-names property is given, the name of the user node will be used as a fallback.

- Drivers for devices that use more than a single PWM device can use the pwm-names property to map the name of the PWM device requested by the pwm_get() call to an index into the list given by the pwms property.

PWM

- The following example describes a PWM-based backlight device, which is an excerpt from the kernel documentation on PWM device binding (see Documentation/devicetree/bindings/pwm/pwm.txt):

```
pwm: pwm {
    #pwm-cells = <2>;
};

[...]

bl: backlight {
pwms = <&pwm 0 5000000>;
   pwm-names = "backlight";
};
```

PWM

- The PWM-specifier typically encodes the chip-relative PWM number and the PWM period in nanoseconds, with the following line:

  pwms = <&pwm 0 5000000>;

- 0 corresponds to the PWM index, relative to the controller, and 5000000 represents the period, in nanoseconds. Note that in the preceding example, specifying the pwm-names is redundant, because the name backlight would be used as a fallback, anyway. Therefore, the driver would have to call the following:

PWM

```
static int my_consummer_probe(struct platform_device *pdev)
{
    struct pwm_device *pwm;


    pwm = pwm_get(&pdev->dev, "backlight");
    if (IS_ERR(pwm)) {
        pr_info("unable to request PWM, trying legacy API\n");
        /*
         * Some drivers use the legacy API as fallback, in order
         * to request a PWM ID, global to the system
         * pwm = pwm_request(global_pwm_id, "pwm beeper");
         */
    }
```

PWM

```
    [...]
    return 0;
}
```

- The PWM specifier typically encodes the chip-relative PWM number and the PWM period, in nanoseconds.

PWM

Using PWMs with the sysfs interface

- The PWM core sysfs root path is /sys/class/pwm/. It is the user space way to manage PWM devices. Each PWM controller/chip added to the system creates a pwmchipN directory entry under the sysfs root path, where N is the base of the PWM chip. The directory contains the following files:
  - npwm: This is a read-only file, printing the number of PWM channels that the chip supports
  - Export: This is a write-only file, allowing you to export a PWM channel for use with sysfs (this functionality is equivalent to the GPIO sysfs interface)
  - Unexport: This unexports a PWM channel from sysfs (write-only)

- The PWM channels are numbered using an index from 0 to pwm<n-1>. These numbers are local to the chip. Each PWM channel exportation creates a pwmX directory in the pwmchipN, which is the same directory as the one containing the export file used. X is the number of the channel that was exported. Each channel directory contains the following files:
  - Period: This is a readable/writable file to get/set the total period of the PWM
  - signal. The value is in nanoseconds.

PWM

- duty_cycle: This is a readable/writable file to get/set the duty cycle of the PWM signal. It represents the active time of the PWM signal. The value is in nanoseconds, and must always be less than the period.
- Polarity: This is a readable/writable file to be used only if the chip of this PWM device supports polarity inversion. It is better to change the polarity only when this PWM is not enabled. Accepted values are string normal or inversed.
- Enable: This is a readable/writable file, to enable (start toggling) or disable (stop toggling) the PWM signal. The accepted values are as follows:
  - 0: disabled
  - 1: enabled

- The following is an example of using PWM from the user space through the sysfs interface:
  - Enable PWM:

    # echo 1 > /sys/class/pwm/pwmchip<pwmchipnr>/pwm<pwmnr>/enable

PWM

- Set the PWM period:

  # echo <value in nanoseconds> >

  /sys/class/pwm/pwmchip<pwmchipnr>/pwm<pwmnr>/period

- Set the PWM duty cycle; the value of the duty cycle must be less than the value of the PWM period:

  # echo <value in nanoseconds> >

  /sys/class/pwm/pwmchip<pwmchipnr>/pwm<pwmnr>/duty_cycle

- Disable PWM:

  # echo 0 > /sys/class/pwm/pwmchip<pwmchipnr>/pwm<pwmnr>/enable

- The complete PWM framework API and sysfs description is available in the Documentation/pwm.txt file, in the kernel source tree.

Thank you